

Name des Projekts:

Einkaufsliste

Datum (TT.MM.JJJJ):

24.04.2026

Vorname und Nachname des Autors:

Nico Seiler

Kursnummer:

21-295-F

M295 & M294 Backend und Frontend für Applikationen realisieren

Backend

1. Projektübersicht

In diesem ÜK-Projekt möchte ich eine einfache Einkaufslisten-App entwickeln. Benutzer können sich registrieren, einloggen und ausloggen. Nach dem Login können sie mehrere Einkaufslisten erstellen und verwalten, sowie Artikel innerhalb dieser Listen erfassen, bearbeiten und als erledigt markieren.

Die App besteht aus einem Java-Backend mit REST-Schnittstellen und einer Datenbank, die alle Daten speichern.

2. Projektziel

Ziel ist es ein funktionierendes Backend zu bauen, das folgende Funktionen hat:

- Benutzer können sich registrieren, einloggen und ausloggen.
- Verwaltung von mehreren Einkaufslisten (erstellen, anzeigen, löschen)
- Verwaltung von Artikeln innerhalb einer Liste (hinzufügen, bearbeiten, löschen)
- Markierung von Artikeln als 'erledigt' oder 'offen'

3. Technische Umsetzung

Technologien

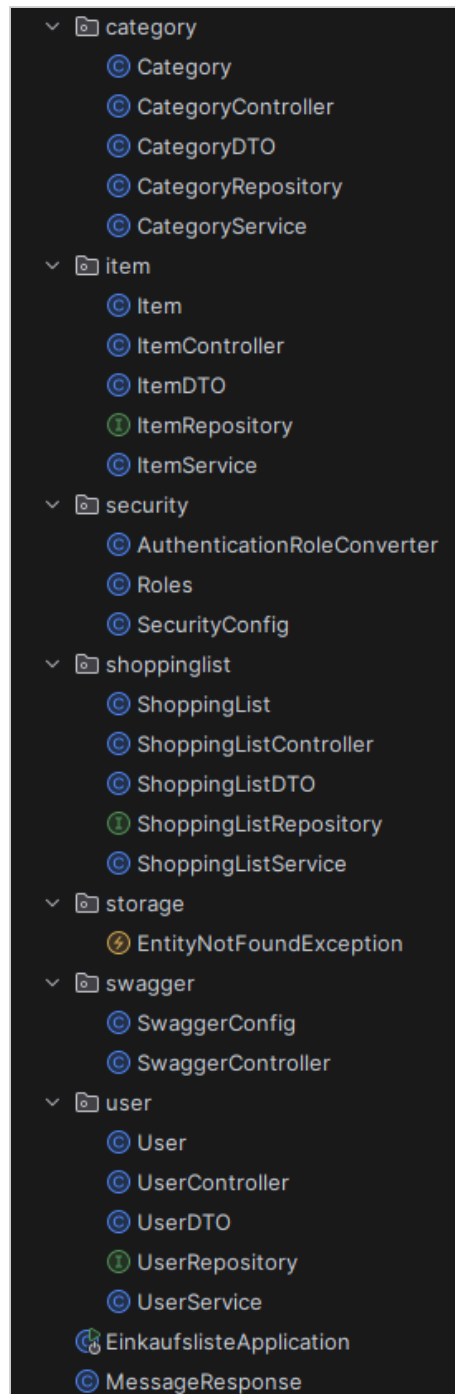
Framework	Spring Boot
Sprache	Java
Datenbank	PostgreSQL mit JPA / Hibernate
Authentifizierung	Keycloak
API-Dokumentation	Swagger UI
Build-Tool	Maven

Projektstruktur

Der Code ist nach Single-Responsibility-Prinzip in folgende Packages gegliedert:

- **item/** - Entity, Repository, Service, Controller für die Artikel
- **category/** - Entity, Repository, Service, Controller für die Kategorien
- **shoppinglist/** - Entity, Repository, Service, Controller für die Einkaufslisten
- **security/** - AuthenticationRoleConverter, Roles, SecurityConfig
- **storage/** - EntityNotFoundException

- **swagger/** - SwaggerConfig, SwaggerController
- **root/** - EinkaufslisteApplication, MessageResponse
- **user/** - Entity, DTO, Repository, Service, Controller für die Benutzer



Entitäten

Die Applikation enthält folgende Entitäten:

- **Category** – id, name, description, color
- **Item** – id, name, note, quantity, done, category
- **ShoppingList** – id, name, description, items, owner
- **User** – id, keycloakId, username, email, firstName, lastName, shoppingLists

REST-Endpunkte

Category

Methode	Endpunkt	Rolle	Beschreibung
GET	api/category	read	Alle Kategorien abrufen
GET	api/category/{id}	read	Kategorie per ID abrufen
POST	api/category	admin	Neue Kategorie erstellen
PUT	api/category/{id}	update	Kategorie aktualisieren
DELETE	api/category/{id}	admin	Kategorie löschen

ShoppingList

Methode	Endpunkt	Rolle	Beschreibung
GET	api/shoppinglist	read	Alle Listen abrufen
GET	api/shoppinglist/{id}	read	Liste per ID abrufen
POST	api/shoppinglist	admin	Neue Liste erstellen
PUT	api/shoppinglist/{id}	update	Liste aktualisieren
DELETE	api/shoppinglist/{id}	admin	Liste löschen

User

Methode	Endpunkt	Rolle	Beschreibung
GET	api/user	admin	Alle Benutzer abrufen
GET	api/user/{id}	read	Benutzer per ID abrufen
GET	api/user/me	read	Eigenes Profil abrufen
PUT	api/user/{id}	admin	Benutzer aktualisieren
DELETE	api/user/{id}	admin	Benutzer löschen

Item

Methode	Endpunkt	Rolle	Beschreibung
GET	api/shoppinglist/{listId}/item	read	Alle Artikel einer Liste abrufen
GET	api/shoppinglist/{listId}/item/{itemId}	read	Artikel per ID abrufen
POST	api/shoppinglist/{listId}/item	admin	Neuen Artikel zur Liste hinzufügen
PUT	api/shoppinglist/{listId}/item/{itemId}	update	Artikel aktualisieren
DELETE	api/shoppinglist/{listId}/item/{itemId}	admin	Artikel löschen
PATCH	api/shoppinglist/{listId}/item/{itemId}/toggle	update	Artikel als erledigt/offen markieren

Sicherheit

Alle REST-Endpunkte sind über OAuth2 mit JWT-Tokens abgesichert. Die Authentifizierung erfolgt via Keycloak. Es werden drei Rollen verwendet:

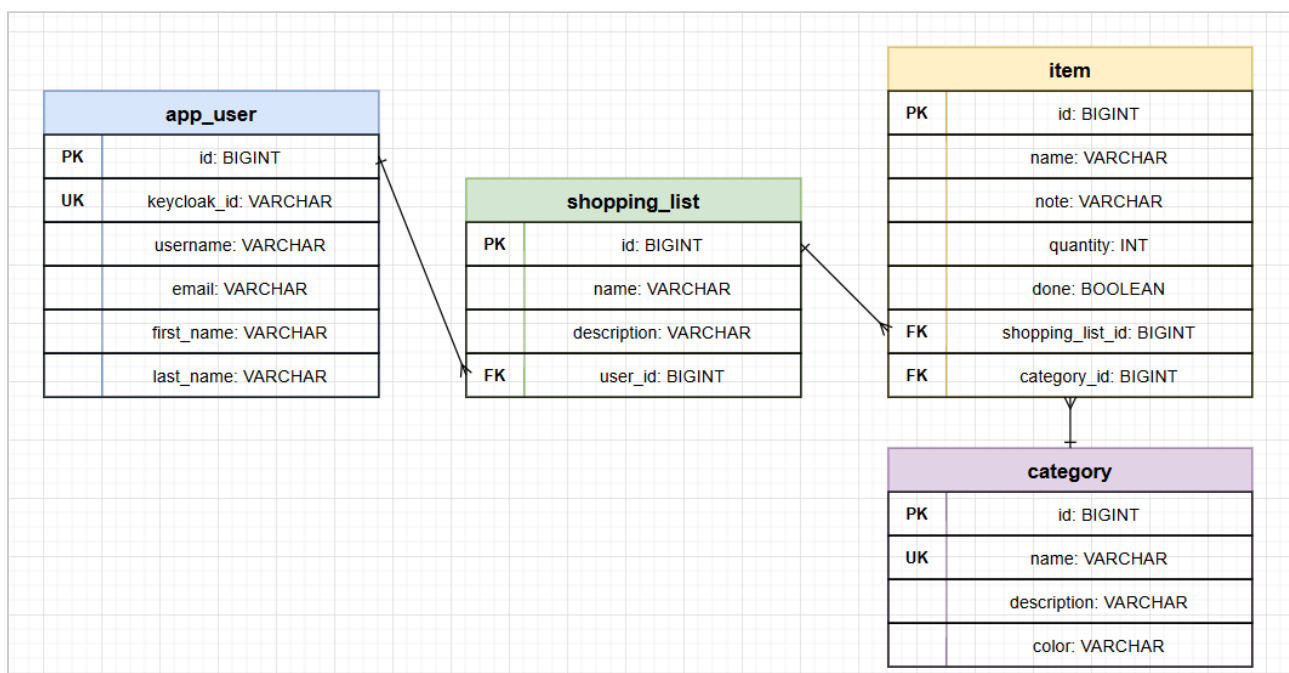
- **read** - Lesezugriff auf alle Ressourcen (GET)
- **update** - Schreibzugriff zum Bearbeiten (PUT/PATCH)
- **admin** - Vollzugriff inkl. Erstellen und Löschen (POST, DELETE)

4. Tests

Für das Projekt wurden JUnit-Tests erstellt:

- **RestControllerTests** – Testet den CategoryController mit Mockito. Alle CRUD-Methoden (getAll, getByld, create, update, delete) werden überprüft.
- **RepositoryTests** – Testet das CategoryRepository über den CategoryService mit Mockito. Alle CRUD-Operationen (insert, find, findAll, update, delete) werden verifiziert.

5. UML Diagramm



Frontend

Beschreibung

Das Frontend der Einkaufsliste ist eine Single-Page-Applikation, die mit Angular (Version 21) und Angular Material umgesetzt wurde. Es kommuniziert über eine REST-Schnittstelle mit dem Spring-Boot-Backend und nutzt Keycloak (über `angular-oauth2-oidc`) für die Authentifizierung und Autorisierung der Nutzer.

Alle Ansichten sind nur für angemeldete Nutzer zugänglich. Je nach Rolle im JWT-Access-Token (**read**, **update**, **admin**) werden dem Nutzer unterschiedliche Funktionen und Menüpunkte angezeigt: Die Rollenprüfung erfolgt sowohl auf Routen-Ebene über einen Auth-Guard als auch im Template über eine eigene strukturelle Direktive (`*appIsInRoles`), die z. B. den Menüpunkt „Benutzerverwaltung“ nur für Nutzer mit der Rolle **admin** einblendet. Ohne gültigen Zugriff wird auf eine „Kein Zugriff“-Seite umgeleitet.

Ansichten

- **Einkaufslisten** (/shoppinglists) – Kartenansicht aller Einkaufslisten des Nutzers. Listen können erstellt, bearbeitet und gelöscht werden; ein Klick auf eine Karte führt zur Detailansicht.
- **Einkaufsliste Detail** (/shoppinglists/:id) – Tabellarische Ansicht aller Artikel einer Liste (Name, Menge, Notiz, Kategorie). Artikel können hinzugefügt, bearbeitet, gelöscht und per Checkbox als erledigt/offen markiert werden.
- **Kategorien** (/categories) – Tabelle aller Kategorien mit Farbe, Name und Beschreibung. Kategorien können erstellt, bearbeitet und gelöscht werden.
- **Benutzerverwaltung** (/users, nur Rolle admin) – Tabelle aller registrierten Benutzer mit der Möglichkeit, Benutzer zu löschen.
- **Mein Profil** (/profile) – Zeigt die Profildaten (Benutzername, Vor- und Nachname) des eingeloggten Nutzers an.
- **Kein Zugriff** (/noaccess) – Hinweisseite, wenn ein Nutzer nicht eingeloggt ist oder die benötigte Rolle fehlt.

Wireframes der wichtigsten Ansichten



← Wocheneinkauf

+ Artikel hinzufügen

☐	Name	Menge	Notiz	Kategorie	Aktionen
☒	Milch	2	1.5%	Milchprodukte	 
☐	Brot	1		Backwaren	 
☐	Äpfel	6	Bio	Obst	 

Abb. 2: Ansicht „Einkaufsliste Detail“ – Artikel-Tabelle mit Erledigt-Checkbox, Kategorie-Chip und Bearbeiten/Löschen je Zeile

Kategorien

+ Kategorie erstellen

Farbe	Name	Beschreibung	Aktionen
	Obst	Frisches Obst	 
	Milchprodukte	Kühlregal	 

Abb. 3: Ansicht „Kategorien“ – Verwaltung der Artikel-Kategorien inkl. Farbe

Benutzer-Anleitung

1. Die Applikation im Browser unter der Frontend-URL öffnen (Entwicklung: `http://localhost:4200`).
2. Über den Login-Button in der Toolbar erfolgt die Anmeldung via Keycloak.
3. Nach dem Login gelangt man über die Navigation zur Übersicht „Einkaufslisten“ mit allen eigenen Listen.
4. Über die Schaltfläche „+“ kann eine neue Einkaufsliste mit Name und Beschreibung erstellt werden.
5. Ein Klick auf eine Listenkarte öffnet die Detailansicht mit allen Artikeln dieser Liste.
6. Über „Artikel hinzufügen“ können Artikel mit Name, Menge, Notiz und Kategorie erfasst werden.
7. Artikel können bearbeitet, gelöscht oder per Checkbox als erledigt/offen markiert werden.
8. Unter „Kategorien“ können eigene Kategorien mit Name, Beschreibung und Farbe verwaltet werden.
9. Nutzer mit der Rolle **admin** sehen zusätzlich den Menüpunkt „Benutzerverwaltung“ und können dort Benutzer löschen.
10. Unter „Mein Profil“ sieht jeder Nutzer seine eigenen Profildaten.
11. Über den Logout-Button in der Toolbar wird die Sitzung beendet.

Konfiguration

Keycloak Realm-Name	Nico-Seiler-Einkaufsliste
Keycloak Client-Name (Client-ID)	einkaufsliste
API-URL	<code>http://localhost:9090/api/</code>
Backend Datenbankname	Nico-Seiler-Einkaufsliste
Backend Datenbank- Port	5432
Backend Server-Port	9090
Frontend-URL	<code>http://localhost:4200</code>

Die Werte für Backend-URL, Keycloak-Realm und -Client sind in den Angular-Environment-Dateien (`environment.ts` / `environment.prod.ts`) sowie in der Auth-Konfiguration (`app.auth.ts`) hinterlegt und können dort bei Bedarf angepasst werden.

Models

Die folgenden TypeScript-Datenmodelle bilden die Backend-Entitäten im Frontend ab:

ShoppingList

- **id** (number) – Eindeutige ID der Liste
- **name** (string) – Name der Einkaufsliste
- **description** (string) – Beschreibung der Liste
- **items** (Item[]) – Artikel innerhalb der Liste

Item

- **id** (number) – Eindeutige ID des Artikels
- **name** (string) – Name des Artikels
- **note** (string) – Notiz zum Artikel
- **quantity** (number) – Menge
- **done** (boolean) – Gibt an, ob der Artikel erledigt ist
- **category** (Category | null) – Zugeordnete Kategorie

Category

- **id** (number) – Eindeutige ID der Kategorie
- **name** (string) – Name der Kategorie
- **description** (string) – Beschreibung der Kategorie
- **color** (string) – Farbe der Kategorie (Hex-Wert)

User

- **id** (number) – Eindeutige ID des Benutzers
- **username** (string) – Benutzername
- **firstName** (string) – Vorname
- **lastName** (string) – Nachname